LatticeGuard: A Multi-Branch Adaptive Defense Framework for Graph Neural Networks

Abhaya Trivedi[0009−0004−7656−4838], Jhalak Gupta[0009−0005−7716−1990], Mehak[0009−0006−9744−8005]⋆

Indira Gandhi Delhi Technical University for Women
{abhaya002btaiml23, jhalak078btaiml23, mehak235btcseai23}@igdtuw.ac.in

Abstract. Graph Neural Networks (GNNs) are increasingly deployed in critical domains, yet their susceptibility to camouflaged poisoning attacks remains a pressing concern. These attacks are difficult to counter because adversarial nodes and edges imitate the statistical patterns of genuine graph data, rendering conventional detection methods ineffective. We introduce LatticeGuard, a scalable defense that unifies four orthogonal strategies into a single end-to-end pipeline: edge pruning to excise overtly malicious links, Bayesian uncertainty estimation via Monte Carlo dropout to quantify prediction robustness, anomaly detection, and randomized smoothing to enhance resilience against camouflaged perturbations. We evaluate LatticeGuard on benchmark citation datasets, including Cora, CiteSeer and PubMed under state-of-the-art camouflaged attackers. LatticeGuard consistently reduces Attack Success Rate (ASR) by up to 56%, while incurring less than 1% clean accuracy drop across canonical base models (GCN and GraphSAGE), and next-generation architectures (GPR and GAT). These results underscore the importance of coordinated, multi-strategy defenses for advancing adversarial robustness in contemporary graph learning systems.

Keywords: Graph Neural Network · Camouflage Poisoning

1 Introduction

Graph Neural Networks (GNNs) are a potent deep learning architecture for analyzing relational data, excelling in applications like fraud detection and citation network classification by effectively capturing complex dependencies [1]. However, their utility in security-sensitive domains is threatened by adversarial data poisoning, where performance degrades due to the injection of malicious nodes or edges [2, 3]. A particularly sophisticated challenge is the camouflaged poisoning attack, which evades standard detection methods by injecting malicious substructures that closely mimic benign graph patterns [4, 5].

Current GNN defense initiatives are often compartmentalized, addressing specific vulnerabilities in isolation [6, 7]. This reliance on localized

⋆ Corresponding author: mehak235btcseai23@igdtuw.ac.in

strategies frequently fails to utilize the synergistic benefits of multiple protection tactics, leaving models susceptible to complex, real-world camouflaged attacks [4]. To address this shortcoming and enhance generalizability and resilience, we introduce LatticeGuard, a novel hybrid defense architecture. Our framework provides a unified, adaptable defense that generalizes effectively across diverse GNN architectures [8] by building upon the core GNN principles of iterative message forwarding and feature aggregation.

2 Related Work

The initial phase of defenses focused on purifying the graph, which included Jaccard similarity based filtering and traditional edge pruning [6]. Although these approaches provided initial resilience, they frequently suffered from over-pruning, sometimes discarding critical structural information, and degrading performance even under benign conditions. To reduce the impact of adversarial nodes or large variance, Bayesian models such as Robust GCN (R-GCN) used uncertainty propagation [4, 9]. Parallel research explored anomaly detection techniques based on embedding distances, clustering, and statistical outlier detection to flag suspicious nodes or edges. These defenses broadened the landscape of graph robustness, but often required careful parameter tuning and could still be bypassed by camouflaged poisoning attacks that closely mimic natural graph distributions [10]. In response, frameworks based on structure learning emerged, most notably ProGNN [11] and its improved variant ProGNN-fs which optimize both the graph structure and the model parameters jointly to resist adversarial perturbations. Although effective, these methods are computationally intensive and often architecture specific, limiting their generalizability across diverse GNNs.

3 Methodology

Our proposed layer-wise defense framework [12, 13] is sensitively designed to mitigate camouflage poisoning attacks [14] by reconstructing a sanitized graph through a strategic integration of statistical and structural defense mechanisms. The framework integrates four complementary modules, each tailored to address distinct aspects of adversarial contamination, collaboratively working to iteratively reinforce graph integrity and enhance model resilience, and effectively neutralizing stealthy perturbations introduced by camouflage poisoning.

3.1 Edge Pruning

Edge pruning is a targeted procedure in our defense framework that aims to remove poorly-connected edges from the graph before and during model training. Its layer-wise integration is designed to minimize the influence of statistically unlikely connections while preserving the structural integrity.

Edges involving flagged nodes, or those with low feature support [15], are removed from the adjacency matrix before being passed to the next layer. The result of edge pruning is essentially allowing finer-grained feature-based refining, incorporating uncertainty and feature smoothness constraints [16].

3.2 Bayesian Prediction

After the edge pruning step produces refined adjacency matrix, Bayesian prediction is performed as a filter. While pruning minimizes obvious pathways for adversarial influence, subtle attacks or unavoidable graph complexity can leave noisy or uncertain nodes even after pruning.

In our approach, we employ Bayesian prediction using Monte Carlo dropout via (2), which allows for multiple stochastic forward passes during inference. This technique approximates Bayesian inference by activating dropout at test time [2]. As a result, it effectively captures both epistemic and aleatoric uncertainty. For each node in the sanitized graph, we perform several randomized forward passes through the GNN while keeping dropout active.

- Softmax Output (Single Pass): For each class c, where $z_{i,c}$ is the logit for node i and class c:

$$\text{Softmax}(z_{i,c}) = \frac{\exp(z_{i,c})}{\sum_{k=1}^{K} \exp(z_{i,k})} \tag{1}$$

- Monte Carlo Dropout (Multiple Pass): For each node, run T stochastic forward passes with dropout:

$$\mathbf{p}_i^{(t)} = \text{Softmax}(\mathbf{z}_i^{(t)}) \quad \text{for } t = 1, \ldots, T \tag{2}$$

  where $\mathbf{p}_i^{(t)}$ is the predicted probability vector for node i at the t-th pass.

- Uncertainty / Entropy: The uncertainty for node i can be measured via entropy of the mean predictive distribution:

$$H(\bar{\mathbf{p}}_i) = -\sum_{c=1}^{K} \bar{p}_{i,c} \log \bar{p}_{i,c} \tag{3}$$

  where $\bar{p}_{i,c}$ is the probability of class c for node i after averaging.

If the entropy is above the threshold, we flag nodes as potentially poisoned for further adversarial scrutiny.

3.3 Anomaly Detection

After the first two layers are implemented, subtle adversarial nodes may persist, especially those carefully camouflaged during the poisoning process. These nodes often manifest as statistical outliers in the latent feature space, due to their atypical connectivities or feature deviations induced by poisoning.

Anomaly detection obtains the embedding vector using the current (defended) GNN. It calculates standard deviation across the embedding dimensions for each node. Anomalous scores are calculated for each node. An anomaly threshold is determined via mean and standard deviation. Nodes with anomalous scores higher than threshold are flagged as anomalous.

3.4 Randomized Smoothing

During inference, for each node, multiple predictions are generated by adding Gaussian noise $\epsilon \sim \mathcal{N}(0, \sigma^2)$ to the node features and passing them through the trained GNN. For a given feature x, the resulting noisy feature is defined as $x_{\text{noisy}} = x + \epsilon$. The predictions from these noisy inputs are collected in a tensor and aggregated by taking the mode along the sample dimension, thereby determining the final class label for each node [17].

The complete LatticeGuard algorithm, detailed in the Appendix A, describes a multi-stage process where each step progressively enhances the robustness of GNNs.

4 Experimental Settings

All tests under this work are conducted using a fully reproducible pipeline implemented in Python, leveraging PyTorch Geometric for graph model construction and training. The code base is modular and compatible with any dataset supported by the Planetoid interface as well as custom graph inputs. The defense mechanisms and poisoning routines utilize only open-source components: NumPy for numeric operations, NetworkX for graph manipulations, and PyTorch for deep learning. No proprietary libraries are involved, ensuring transparency and ease of adaptation.

The architecture is flexible, defense routines can be applied across diverse GNN backbones [18, 19] and extended to directed or undirected graphs with minor changes. For all baselines, experimental conditions were strictly controlled to ensure fair and reproducible comparisons across models for their relative robustness and performance under varying poisoning scenarios.

4.1 Poisoning Strategy

The primary adversarial scenario is a camouflage poisoning attack. Here, a specified fraction of fake (adversarial) nodes is synthesized by drawing features from the global distribution of real node features. These nodes are preferentially linked to a target class, simulating camouflage connectivity that mimics the benign graph structure and making detection difficult [10, 5]. Poisoning rates ranged from 1% to 80% of the total node count, allowing the evaluation of low to extreme attack intensity.

The attack was implemented using the apply_poisoning routine, which first generates new nodes with features sampled from the mean and standard deviation empirical characteristics of the dataset. Then it establishes edges predominantly between fake nodes and a random subset of nodes belonging to the target class. Later, it appends poisoned nodes and edges to the graph, adjusts labels, and ensures training masks include all poisoned samples. The same is portrayed by Fig 1.

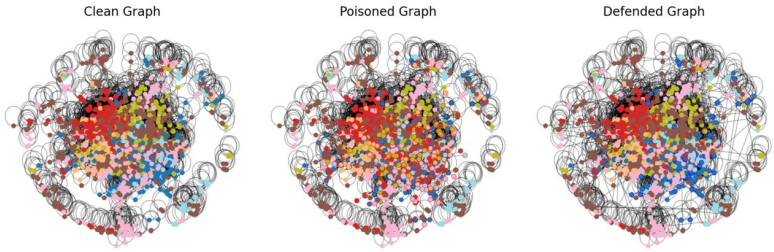

Fig. 1: Illustrates Graph States under Adversarial Conditions.

4.2 Training and Hyperparameters

All GNN classifiers were trained via stochastic optimization of the cross-entropy loss.

$$\mathcal{L}_{\text{train}} = -\sum_{i \in \text{train}} \log p_{y_i} \tag{4}$$

We train the model using Adam optimizer with a learning rate of $\eta = 0.01$ and weight decay $\lambda = 5 \times 10^{-4}$. The model uses hidden size layers $h = 16$, and dropout with rate $p = 0.5$ is applied after each hidden activation to regularize training:

$$\mathbf{X}' = \text{Dropout}\big(\text{ReLU}(W\mathbf{X})\big) \tag{5}$$

GNNs and MLPs were trained for up to 200 and 100 epochs respectively, with early stopping based on validation loss. For each poisoning fraction and defense method, we tracked compute time, memory usage, and CPU load. Additionally, experiments were repeated using Monte Carlo dropout or randomized smoothing with T stochastic forward passes to estimate the predictive mean μ, standard deviation σ, and model uncertainty. The tuning pipeline is delineated in Fig. 2.

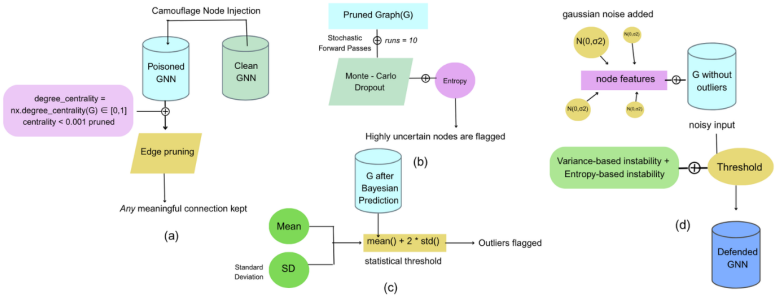

Fig. 2: An overview of LatticeGuard.

5 Results

We conducted a detailed empirical study of the proposed hybrid defense framework on three citation network datasets - Cora, CiteSeer, and PubMed, comparing it against four baseline architectures: GCN, GraphSAGE, GPR, and GAT. Evaluation metrics included classification accuracy, attack success rate (ASR), and computational efficiency.

The standard GCN achieved a baseline accuracy of 79.5% on clean data, but under 20% camouflage poisoning, the accuracy dropped dramatically to 64.3% as depicted in Fig. 3, revealing vulnerability to subtle adversarial perturbations. Applying our hybrid defense significantly recovered performance, boosting post-attack accuracy to 80.5%, demonstrating robust adversarial mitigation while maintaining high predictive power.

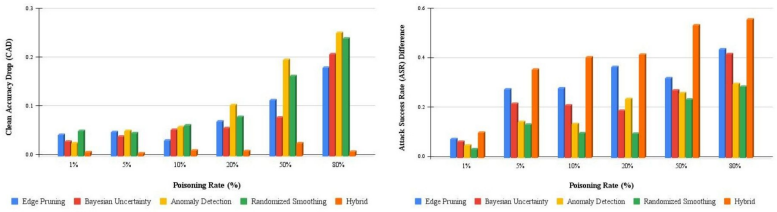

Fig. 3: Variation in CAD and ASR for elementary defense strategies.

The Attack Success Rate (ASR) measures the proportion of target nodes successfully misclassified due to poisoning. Unprotected, poisoned model recorded an ASR of 74.5%, which dropped drastically to 20.7% after applying LatticeGuard, demonstrating effective misclassification neutralization. The study outcomes are reported in Table 1.

In addition to robustness, computational efficiency is critical for practical deployment. Our hybrid defense mechanism completes the inference in about 16 seconds with a peak memory usage of 942 MiB and

Table 1: Comparative Evaluation of Defense Mechanisms

Poisoning %	Clean Acc	Poisoned Acc	ASR	Edge Pruning		Bayesian Uncertainty		Anomaly Detection		Randomized Smoothing		Hybrid Defense	
				Acc	ASR	Acc	ASR	Acc	ASR	Acc	ASR	Acc	ASR
1%	0.795	0.743	0.392	0.752	0.316	0.765	0.327	0.769	0.342	0.743	0.357	0.787	0.290
5%	0.795	0.710	0.606	0.746	0.329	0.755	0.388	0.744	0.460	0.748	0.471	0.790	0.249
10%	0.795	0.702	0.638	0.763	0.357	0.741	0.426	0.735	0.501	0.732	0.538	0.784	0.231
20%	0.795	0.643	0.676	0.724	0.308	0.738	0.485	0.691	0.437	0.715	0.579	0.805	0.258
50%	0.795	0.545	0.745	0.680	0.423	0.716	0.473	0.598	0.482	0.631	0.508	0.821	0.207
80%	0.795	0.487	0.823	0.613	0.384	0.586	0.402	0.542	0.523	0.554	0.534	0.804	0.263

near-maximal CPU utilization (around 99%), reflecting efficient resource use. Individual components execute faster, taking roughly 4 seconds with slightly higher memory usage (973 MiB). This balance between runtime and memory footprint underscores the practical viability of our solution.

Our hybrid defense mechanism demonstrates strong compatibility with all four baseline models across diverse datasets and poisoning rates. Comparative analysis (Fig. 4) reveals that it generalizes well, adapts to structural variations in heterogeneous networks, and maintains high stability under challenging conditions. It notably outperforms models like GCN and GAT, effectively preventing catastrophic accuracy drops under high poisoning rates on the PubMed dataset, as shown in Table 2.

Table 2: Performance across Baseline Architectures

(%)	SAGE			GCN			GAT			GPR		
	C	CS	P	C	CS	P	C	CS	P	C	CS	P
1	0.786	0.662	0.254	0.787	0.678	0.433	0.804	0.629	0.279	0.790	0.633	0.403
5	0.799	0.643	0.690	0.790	0.656	0.483	0.772	0.656	0.707	0.768	0.588	0.672
10	0.792	0.642	0.760	0.784	0.651	0.740	0.787	0.643	0.800	0.797	0.630	0.740
20	0.796	0.649	0.756	0.806	0.631	0.650	0.817	0.613	0.585	0.818	0.591	0.875
50	0.799	0.570	0.182	0.822	0.582	0.429	0.808	0.552	0.417	0.817	0.563	0.409
80	0.826	0.553	0.012	0.804	0.563	0.091	0.793	0.589	0.100	0.820	0.602	0.001

Note: C - Cora, CS - CiteSeer, P - PubMed.

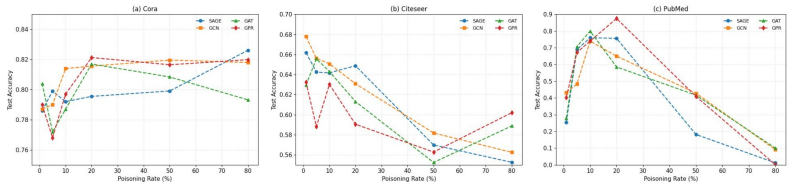

Fig. 4: Test Accuracy of SAGE, GCN, GAT, and GPR under varying Poisoning Levels on Cora, CiteSeer, and PubMed Datasets.

When our hybrid defense is benchmarked against prominent models such as GCN-Jaccard [20], Pro-GNN [11], and Pro-GNN-fs, our method consistently performed equivalent or better across varying poisoning rates, as presented in Table 3. For instance, at 80% poisoning, the hybrid achieves an accuracy of up to 0.83, surpassing both the GCN-Jaccard and Pro-

GNN variants. These findings confirm that the integrated approach provides competitive advantage and generalizes better.

Table 3: Benchmarking against state-of-the-art models

Poisoning Rate	Pro-GNN	Pro-GNN-fs	GCN-Jaccard	LatticeGuard
0	0.83	0.82	0.81	0.78
10	0.83	0.82	0.81	0.78
20	0.82	0.81	0.80	0.80
50	0.81	0.80	0.78	0.82
80	0.80	0.77	0.75	0.83

Our results demonstrate that the proposed hybrid defense mechanism significantly improves resilience against camouflaged poisoning attacks without degrading the performance on clean data.

6 Conclusion and Future Work

The proposed hybrid defense mechanism offers a holistic approach for protecting Graph Neural Networks (GNNs) against camouflaged poisoning attacks. It delivers robust adversarial resilience by dramatically decreasing the ASR and has the ability to increase or maintain clean accuracy through a wide range of poisoning levels. Overall, the hybrid defense mechanism proves the potential to not just remove malicious perturbations but also improve generalization.

Although hybrid defense demonstrated multiple strengths, there are still several trade-offs to consider. Performing multiple defense processes has increased the computational cost and design may present scalability concerns, particularly for highly dynamic networks. It is also important to consider the potential risk that aggressively pruning edges or sensitive anomaly detection thresholds would incorrectly identify legitimate, but unusual attributes. This could affect graph connectivity in the real world.

We have not even yet been able to explore a range of concepts to extend hybrid defense to even larger graphs with millions of nodes and edges, without exploitable time or memory costs. Another key challenge is scaling this method for other large-scale adversarial attacks. We can pursue approaches such as mini-batch processing to aid in anomaly detection, parallelization of pruning algorithms, and the incremental learning approach.

References

1. Wang, J., Song, G., Wu, Y., Wang, L.: Streaming graph neural networks via continual learning. In: CIKM '20: Proceedings of the 29th

ACM International Conference on Information & Knowledge Management, pp. 1515–1524. ACM, New York (2020)
2. Islam, M.F., Zabeen, S., Rahman, F.B., Islam, M.A., Kibria, F.B., Manab, M.A., Karim, D.Z., Rasel, A.A.: Exploring node classification uncertainty in graph neural networks. In: ACMSE, pp. 186–190 (2023)
3. Zhu, D., Zhang, Z., Cui, P., Zhu, W.: Robust graph convolutional networks against adversarial attacks. In: KDD '19: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 1399–1407. ACM, New York (2019)
4. Zügner, D., Günnemann, S.: Certifiable robustness and robust training for graph convolutional networks. In: KDD '19: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 1–11. ACM, New York (2019)
5. Di, J.Z., Douglas, J., Acharya, J., Kamath, G., Sekhari, A.: Hidden poison: Machine unlearning enables camouflaged poisoning attacks. In: 37th Conference on Neural Information Processing Systems (NeurIPS 2023), pp. 1–27. Curran Associates, Inc., New Orleans (2023)
6. Zhang, Y., Pal, S., Coates, M., Üstebay, D.: Bayesian graph convolutional neural networks for semi-supervised classification. In: AAAI '19: Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence, vol. 33, pp. 5829–5836. AAAI Press (2019)
7. Zügner, D., Günnemann, S.: Adversarial attacks on graph neural networks via meta learning. In: 7th International Conference on Learning Representations (ICLR 2019), pp. 1–16. ICLR, New Orleans (2019)
8. Wang, X., Liu, X., Hsieh, C.-J.: GraphDefense: Towards robust graph convolutional networks. arXiv preprint arXiv:1911.04429 (2019)
9. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. In: 5th International Conference on Learning Representations (2017)
10. Jiang, C., He, Y., Chapman, R., Wu, H.: Camouflaged poisoning attack on graph neural networks. In: ICMR '22: Proceedings of the 2022 International Conference on Multimedia Retrieval, pp. 451–461. ACM, New York (2022)
11. Jin, W., Ma, Y., Liu, X., Tang, X., Wang, S., Tang, J.: Graph structure learning for robust graph neural networks. In: 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD '20), pp. 1–10. ACM, New York (2020).
12. Hamilton, W.L., Ying, R., Leskovec, J.: Inductive representation learning on large graphs. In: 31st Conference on Neural Information Processing Systems (2017)
13. Miao, X., Zhang, W., Shao, Y., Cui, B., Chen, L., Zhang, C.: Lasagne: A multi-layer graph convolutional network framework via node-aware deep architecture. IEEE Trans. Knowl. Data Eng. **35**(2), 1721–1733 (2023)
14. Zhang, X., Zitnik, M.: GNNGuard: Defending graph neural networks against adversarial attacks. In: 34th Conference on Neural Informa-

tion Processing Systems (NeurIPS 2020), pp. 1–13. Curran Associates, Inc., Vancouver (2020)
15. Zhu, Y., Huang, J., Chen, Y., Amor, R., Witbrock, M.: A graph transformer defence against graph perturbation by a flexible-pass filter. Inf. Fusion **107**(C) (2024). https://doi.org/10.1016/j.inffus.2024.102296
16. Kato, H., Hasegawa, K., Hidano, S., Fukushima, K.: EdgePruner: Poisoned edge pruning in graph contrastive learning. In: 2024 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), pp. 1–15. IEEE (2024)
17. Rosenfeld, E.: Generalizing randomized smoothing for pointwise-certified defenses to data poisoning attacks. ML@CMU Blog, Carnegie Mellon University (2020). https://blog.ml.cmu.edu/2020/09/25/generalizing-randomized-smoothing/
18. Jin, D., Yu, Z., Huo, C., Wang, R., Wang, X., He, D., Han, J.: Universal graph convolutional networks. In: 35th Conference on Neural Information Processing Systems (2021)
19. Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y.: Graph attention networks. In: 6th International Conference on Learning Representations (2018)
20. Wu, H., Wang, C., Tyshetskiy, Y., Docherty, A., Lu, K., Zhu, L.: Adversarial examples on graph data: Deep insights into attack and defense. In: 28th International Joint Conference on Artificial Intelligence (IJCAI '19), pp. 4816–4823. ijcai.org (2019)
21. Gal, Y.: Uncertainty in Deep Learning. PhD Thesis, University of Cambridge, UK (2016)
22. Chen, Z., Wu, Z., Sadikaj, Y., Plant, C., Dai, H.N., Wang, S., Guo, W.: ADEdgeDrop: Adversarial edge dropping for robust graph neural networks. arXiv preprint arXiv:2403.09171 (2024)
23. Gal, Y., Ghahramani, Z.: Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In: Balcan, M.F., Weinberger, K.Q. (eds.) ICML 2016, PMLR, vol. 48, pp. 1050–1059. JMLR.org (2016)

A Appendix

Algorithm 1 Framework Mechanism

Require: Graph $G = (A, X, Y)$, defense parameters (τ, σ, T)

$G' \leftarrow \text{AddSelfLoops}(G)$ ▷ Add self-loops
$DC \leftarrow \text{DegreeCentrality}(G')$ ▷ Find Degree centrality
$V_{\text{keep}} \leftarrow \{v \mid DC[v] > \tau\}$ ▷ Prune low-centrality nodes
$G_{\text{pruned}} \leftarrow \text{Subgraph}(G, V_{\text{keep}})$
RestoreConnectivity(G_{pruned}) ▷ Add edges to connect components
GNN $\leftarrow$ InitializeGNN()
Train(GNN, G_{pruned})
MLP $\leftarrow$ InitializeMLP()
Train(MLP, $X_{\text{pruned}}, Y_{\text{pruned}}$)
preds $\leftarrow$ [ ]
for $t = 1$ to T **do**
    GNN.train() ▷ Enable dropout
    pred $\leftarrow$ Softmax(GNN($X_{\text{pruned}}, A_{\text{pruned}}$))
    preds.append(pred)
end for
$\mu, \sigma_p \leftarrow$ Mean, Std(preds)
emb $\leftarrow$ GNN($X_{\text{pruned}}, Y_{\text{pruned}}$)
score $\leftarrow$ Std(emb, dim=1)
$\tau_a \leftarrow$ Mean(score) + 2 × Std(score)
$V_{\text{anom}} \leftarrow \{v \mid \text{score}[v] > \tau_a\}$
smooth_preds $\leftarrow$ [ ]
for $s = 1$ to T **do**
    $X_{\text{noisy}} \leftarrow X_{\text{pruned}} + \sigma \times \mathcal{N}(0, I)$
    pred $\leftarrow$ Argmax(GNN($X_{\text{noisy}}, A_{\text{pruned}}$))
    smooth_preds.append(pred)
end for
$\hat{y}_{\text{smooth}} \leftarrow$ Mode(smooth_preds)
$\hat{y}_{\text{gnn}} \leftarrow$ Argmax(GNN($X_{\text{pruned}}, A_{\text{pruned}}$))
$\hat{y}_{\text{mlp}} \leftarrow$ Argmax(MLP(X_{pruned}))
for each node $v \in V_{\text{keep}}$ **do**
    if $\hat{y}_{\text{gnn}}[v] = \hat{y}_{\text{mlp}}[v]$ **then**
        $\hat{y}_{\text{final}}[v] \leftarrow \hat{y}_{\text{gnn}}[v]$
    else
        $\hat{y}_{\text{final}}[v] \leftarrow \hat{y}_{\text{smooth}}[v]$
    end if
end for
return $\hat{y}_{\text{final}}$